# Day 7: Weekly Revision

## 1. Day Number

**Day 7**

## 2. Topic Name

**Revision of Days 1–6**

Today you will revise:

- Variables
- Input and output
- Strings and numbers
- Type conversion
- Comparison operators
- Arithmetic operators
- `if-else`
- Nested `if`
- Basic input validation

## 3. Connection

During the previous days, you learned how to:

1. Store a password.
2. Ask the user to enter the password.
3. Check whether the password is correct.
4. Create simple bank account information.
5. Deposit money.
6. Validate banking amounts.

Today, you will combine **password validation** with **one simple banking action**.

The program will first verify the password. Only after successful login will it allow the user to deposit money.


---

## 4. Revision Summary of Days 1–6

### Day 1: Store Password

You learned how to save information inside a variable.

```
saved_password = "python123"
```

Here:

- `saved_password` is the variable name.
- `"python123"` is a string value.

### Day 2: Validate Password

You learned how to compare two values using ==.

```
entered_password == saved_password
```

This comparison produces a Boolean result:

```
True
```

or:

```
False
```

### Day 3: Conditions and Nested Decisions

You learned how a program makes decisions using:

```
if
else
```

You also learned that one condition can be placed inside another condition. This is called a **nested condition**.

```
If login is successful:
    Check the banking input
```

### Day 4: Open a Bank Account

You learned about different data types:

- Account holder name → string
- Account number → string or integer
- Account balance → integer or float

You also learned type conversion:

```
float(user_input)
```

### Day 5: Deposit Amount

You learned how to add money to a balance.

```
balance = balance + deposit
```

The shorter version is:

```
balance += deposit
```

### Day 6: Validate a Banking Amount

You learned that banking input should be checked before changing the balance.

For example:

```
Deposit greater than 0 → accept it
Deposit equal to or less than 0 → reject it
```

---

## 5. Important Topics

### Variables

Variables store information.

```
saved_password = "bank123"
balance = 500.0
```

### Input

`input()` receives information from the user.

```
entered_password = input("Enter password: ")
```

Remember that `input()` returns a **string**.

### Output

`print()` displays information.

```
print("Login successful")
```

### Type Conversion

A deposit amount must be converted from a string into a number.

```
deposit = float(input("Enter deposit amount: "))
```

#### `if-else`

Conditions control which part of the program runs.

```
if condition:
    # Run when condition is True
else:
    # Run when condition is False
```

### Nested `if`

A condition can be written inside another condition.

```
if password_is_correct:
    if deposit_is_valid:
        # Update balance
```

### Operators

| Operator | Meaning | Example |
|---|---|---|
| `==` | Equal to | `password == saved_password` |
| `>` | Greater than | `deposit > 0` |
| `+` | Addition | `balance + deposit` |
| `+=` | Add and update | `balance += deposit` |

---

## 6. Foundational Notes

### Note 1: Authentication must happen first

The banking action should only be available after the password is verified.

Incorrect order:

```
Ask for deposit
Then check password
```

Correct order:

```
Check password
Then ask for deposit
```

### Note 2: Password comparison is case-sensitive

These passwords are different:

```
Bank123
bank123
```

Python treats uppercase and lowercase letters differently.

### Note 3: User input is initially text

Even when the user enters:

```
100
```

`input()` receives it as:

```
"100"
```

You must convert it before performing arithmetic.

### Note 4: Validate before updating

Do not immediately add the deposit to the balance.

First check:

```
Is the deposit greater than 0?
```

Only then update the balance.

### Note 5: A deposit should be positive

A valid deposit increases the balance.

```
Deposit > 0
```

A deposit of `0` does not change the balance, while a negative deposit incorrectly reduces it.

---

## 7. Easy Example

Suppose the program contains:

```
Saved password: bank123
Current balance: $500
```

The user enters:

```
Password: bank123
Deposit: $100
```

Because the password is correct and the deposit is positive:

```
Login successful
Deposit successful
Updated balance: $600
```

Another example:

```
Password: wrong123
```

Output:

```
Access denied
```

The program should not ask for a deposit after an incorrect password.

---

## 8. Revision Problem Statement

Create a simple Python banking program.

The program should:

1. Store a password.

2. Store an initial account balance.
3. Ask the user to enter the password.
4. Compare the entered password with the saved password.
5. If the password is correct:

- Display a login-success message.
- Ask the user for a deposit amount.
- Convert the deposit amount into a number.
- Accept the deposit only when it is greater than `0`.
- Add the valid deposit to the balance.
- Display the updated balance. 6. If the password is incorrect:
- Display `"Access denied"`.
- Do not allow the deposit action.

---

## 9. Concepts Used

This problem uses:

- Variables
- Strings
- Numeric values
- `input()`
- `print()`
- `float()`
- Comparison using ==
- Validation using >
- Arithmetic using + or +=
- `if-else`
- Nested `if`
- Boolean thinking
- Function definition
- Function call
- Returning or updating a value

---

## 10. Thought Process

Think about the problem in small steps.

### Step 1: Store the starting information

The program needs:

- A saved password
- An initial balance

```
saved password = bank123
balance = 500
```

### Step 2: Ask for the password

Store the user's entry in another variable.

```
entered password = user input
```

### Step 3: Compare the passwords

Ask:

```
Is entered password equal to saved password?
```

### Step 4: Handle a wrong password

When the comparison is false:

```
Display access denied
```

Do not continue to the deposit step.

### Step 5: Handle a correct password

When the comparison is true:

```
Display login successful
Ask for deposit amount
```

### Step 6: Convert the deposit

The deposit arrives as a string, so convert it into a number.

```
deposit = converted user input
```

### Step 7: Validate the deposit

Ask:

```
Is deposit greater than 0?
```

### Step 8: Update the balance

For a valid deposit:

```
new balance = old balance + deposit
```

### Step 9: Display the result

Show the updated balance.

For an invalid deposit, show an appropriate message without changing the balance.

---

## 11. Pseudocode

```
START

CREATE a function for depositing money
    ASK user for deposit amount
    CONVERT deposit amount to a number

    IF deposit amount is greater than 0
        ADD deposit amount to balance
        DISPLAY deposit successful
        DISPLAY updated balance
    ELSE
        DISPLAY invalid deposit

    RETURN balance

CREATE a main function
    SET saved password
    SET initial balance

    ASK user to enter password

    IF entered password equals saved password
        DISPLAY login successful
        CALL deposit function
    ELSE
        DISPLAY access denied

CALL main function

END
```

---

## 12. Suggested Solving Approach: Functional Approach

Instead of writing everything in one large block, divide the program into functions.

A simple structure could be:

```
deposit_money(balance)
    Handles deposit input, validation and balance update

main()
    Stores account information
    Checks the password
    Calls deposit_money() after successful login
```

Possible structure:

```
def deposit_money(balance):
    # Ask for and validate deposit
    # Return updated balance

def main():
    # Store password and balance
    # Validate login
    # Call deposit function when login succeeds
```

Benefits of functions:

- The program becomes easier to read.
- Each function has one clear responsibility.
- Errors are easier to find.
- The deposit logic can be reused later.
- Future actions such as withdrawal can be added more easily.

---

## 13. Easy Edge Cases

### Wrong Password

Input:

```
Saved password: bank123
Entered password: wrong123
```

Expected behaviour:

```
Access denied
```

The program must not ask for a deposit.

### Negative Deposit

Input:

```
Deposit: -100
```

Expected behaviour:

```
Invalid deposit amount
```

The balance must remain unchanged.

### Deposit of `0`

Input:

```
Deposit: 0
```

Expected behaviour:

```
Invalid deposit amount
```

The balance must remain unchanged.

### Uppercase or Lowercase Difference

```
Saved password: bank123
Entered password: Bank123
```

Expected behaviour:

```
Access denied
```

### Empty Password

Input:

```
Entered password:
```

Expected behaviour:

```
Access denied
```

---

## 14. Common Mistakes to Avoid

### Comparing with = instead of ==

Incorrect:

```
if entered_password = saved_password:
```

Correct comparison operator:

```
if entered_password == saved_password:
```

= assigns a value, while == compares two values.

### Forgetting type conversion

Incorrect:

```
deposit = input("Enter deposit: ")
balance = balance + deposit
```

Here, `deposit` is a string and cannot be directly added to a number.

### Updating before validation

Incorrect thinking:

```
Add deposit to balance
Then check whether deposit was valid
```

Correct thinking:

```
Validate deposit
Then update balance
```

### Allowing a deposit after failed login

The deposit function should only be called inside the successful-login condition.

### Accepting zero or negative deposits

A valid deposit condition should check that the amount is strictly greater than zero.

### Forgetting to return the updated balance

When a function updates a value, make sure the new value is returned or otherwise handled correctly.

### Incorrect indentation

Python uses indentation to identify which statements belong to an `if`, `else`, or function.

```
if condition:
    print("Inside condition")

print("Outside condition")
```

---

## 15. Quick Self-Check Questions

1. What is the difference between = and ==?
2. Why must the deposit input be converted using `float()` or `int()`?
3. Why should the deposit function only run after successful password validation?
4. What should happen to the balance when the deposit is `0` or negative?
5. Why is an `if` statement inside another `if` statement called a nested condition?

---

## 16. Hint Only

Create two functions:

```
def deposit_money(balance):
    ...

def main():
    ...
```

Inside `main()`:

```
Compare the entered password with the saved password.
```

Inside `deposit_money()`:

```
Convert the deposit to a number.
Check whether it is greater than zero.
Only then add it to the balance.
```

Use this condition pattern:

```
if entered_password == saved_password:
    # Login successful
    # Call deposit function
else:
    # Access denied
```

Then place another condition inside the deposit function to validate the amount.